

Práctica PAI-4
27 de abril de 2026

PAI-4 - DevSecOps - ASEGURAMIENTO DE LA CADENA DE SUMINISTRO EN EL DESARROLLO DE APLICACIONES WEB

Proyecto Charmaway
<https://github.com/PAI-4-Charmaway/PAI-4-Charmaway>



Miembros del equipo:
Manuel Jesús Niza Cobo
José Egido Carnerero
Andrés Ponga González

1. Informe técnico

1.1 Contexto y alcance

Charmaway es una aplicación e-commerce desarrollada en Django 5.2 con persistencia en PostgreSQL, integración de pagos Stripe, envío de correo vía Mailjet y despliegue en Render. El alcance de la práctica PAI-4 consiste en dotar a su ciclo de integración continua de capacidades DevSecOps mediante la integración de herramientas de testeo de seguridad en cada fase del pipeline y la centralización de las vulnerabilidades detectadas en una plataforma de gestión, completando un ciclo trazable detección → ticketing → corrección → verificación.

1.2 Pipeline CI/CD

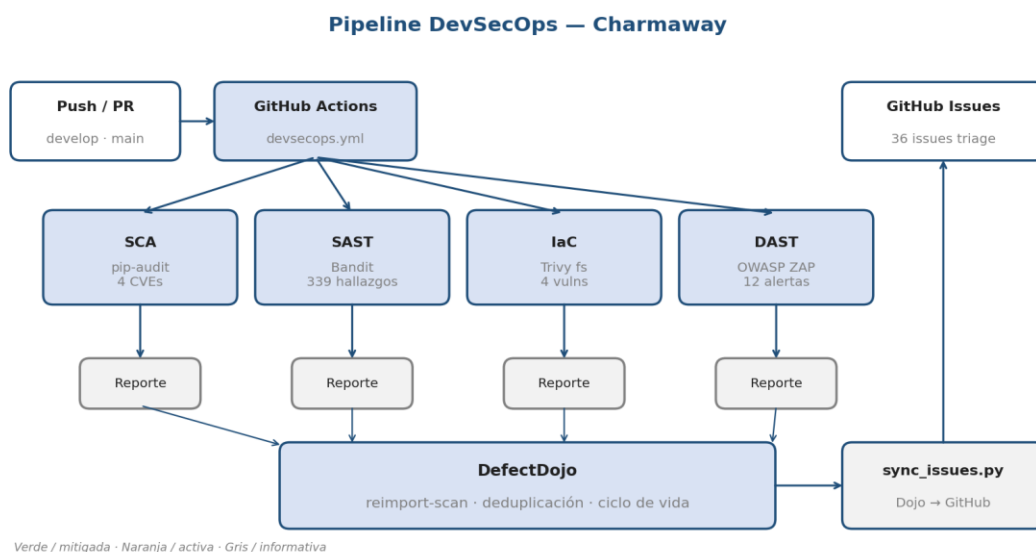


Figura 1. Arquitectura del pipeline DevSecOps de Charmaway.

Se ha implementado un único workflow de GitHub Actions ubicado en `.github/workflows/devsecops.yml` que se dispara automáticamente con cada push a las ramas `develop` y `main`, así como manualmente mediante `workflow_dispatch`. El pipeline está compuesto por cuatro jobs independientes que se ejecutan en paralelo, lo que reduce el tiempo total de ejecución a menos de dos minutos por commit.

Cada job sigue el mismo patrón de tres pasos: (1) ejecución de la herramienta de análisis con generación de reporte en JSON o XML, (2) publicación del reporte como artifact descargable de GitHub, y (3) envío del reporte a la API de DefectDojo mediante el endpoint `/api/v2/reimport-scan/`, lo que permite el versionado y la deduplicación automática de findings entre ejecuciones.

1.3 Herramientas de seguridad seleccionadas

Se han integrado cuatro herramientas que cubren cuatro de las cinco categorías sugeridas en el enunciado, superando el mínimo de tres exigido. Se ha priorizado la cobertura de la cadena de suministro, el código fuente, la infraestructura como código y el comportamiento dinámico de la aplicación en ejecución.

Fase	Herramienta	Tipo de análisis	Justificación
SCA	pip-audit	Dependencias Python	Mantenida por la PyPA. Consume las advisories del Python Packaging Advisory Database y de GitHub Security Advisory. Sin necesidad de registro ni clave.
SAST	Bandit	Código fuente Python	Estándar de facto para auditoría estática de Python. Detecta uso inseguro de aleatoriedad, contraseñas hardcoded, SQL injection y assertions, entre otros.

Fase	Herramienta	Tipo de análisis	Justificación
IaC	Trivy (modo fs)	Infraestructura y secretos	Soporta múltiples formatos (Dockerfile, docker-compose, manifiestos) y combina escaneo de configuración, vulnerabilidades de SO y detección de secretos en un único binario.
DAST	OWASP ZAP Baseline	Aplicación en ejecución	Acción oficial action-baseline mantenida por la Fundación OWASP. Lanza un escaneo pasivo contra la aplicación servida en localhost:8080 y exporta el reporte en XML compatible con DefectDojo.

1.4 Gestión centralizada de vulnerabilidades

Se ha desplegado DefectDojo como herramienta de gestión, agregación y triage de los findings producidos por las cuatro herramientas anteriores. El despliegue se realiza mediante un docker-compose propio (defectdojo/docker-compose.yml) compuesto por siete servicios: postgres, redis, initializer, uwsgi, celeryworker, celerybeat y nginx. La instancia se expone en el puerto 8080 y se inicializa con las credenciales por defecto admin/admin.

Cada job del pipeline sube su reporte al producto Charmaway, dentro de un engagement específico por fase (CI - SCA, CI - SAST, CI - IaC, CI - DAST). El uso del endpoint reimport-scan en lugar de import-scan garantiza que los findings ya conocidos se deduplican en lugar de crearse de nuevo, y los que han desaparecido en la nueva ejecución se marcan automáticamente como Mitigated, lo que permite rastrear el ciclo de vida completo de cada vulnerabilidad.

Como complemento al panel se ha desarrollado el script scripts/sync_issues.py, que consulta los findings activos vía API REST y crea automáticamente un issue en GitHub por cada uno, etiquetando el finding en DefectDojo con el tag enviado-a-github para evitar duplicados en sucesivas ejecuciones. De este modo cada vulnerabilidad queda trazada en el mismo SCM donde se gestiona el código.

1.5 Resultados de la última ejecución

La siguiente tabla resume los hallazgos detectados por cada herramienta en la última ejecución completa del pipeline (workflow run 24995850674).

Fase	Findings	Distribución	Estado
SCA	4 CVEs	Medium en pytest, requests, python-dotenv, sqlparse	2 mitigadas, 2 activas
SAST	339 hallazgos	B101 assert (291), B106 password (37), B311 random (9), B105 (2). Todos LOW.	Se han mitigado las B106, las demás siguen activas
IaC	4 vulnerabilidades	Mismas dependencias detectadas también por Trivy fs.	Coinciden con SCA
DAST	12 alertas	2 Medium, 5 Low, 5 Informational. CSP, SRI, COEP, CORP, etc.	Parcialmente mitigadas

Como prueba del flujo end-to-end, durante esta práctica se han mitigado dos CVEs concretos detectados por la fase SCA mediante actualización dirigida de las dependencias afectadas:

Dependencia	Inicial	Fijada	Identificador y descripción
python-dotenv	1.2.1	1.2.2	CVE-2026-28684 — Symlink overwrite en set_key/unset_key
sqlparse	0.5.3	0.5.4	GHSA-27jp-wm6q-gp25 — ReDoS al formatear tuplas largas

Tras el merge de los fix a main (PR #57 a develop y PR #58 a main, ambos con rebase merge para preservar historial individual), el reimport del nuevo reporte de pip-audit en DefectDojo marcó los dos findings como Mitigated con timestamp, mientras que los dos restantes (CVE-2025-71176 en pytest y CVE-2026-25645 en requests) permanecen activos y se han documentado para una iteración posterior por requerir un major bump. Se pueden encontrar evidencias de las Issues resueltas de SAST y de DAST por todos los miembros en la carpeta “evidencias”, por ejemplo, se puede mirar a los archivos “cap-issues”, “cap-local-issues-josegicar” o “cap-local-issues-andres”. Junto a esto, se pueden ver las Issues Cerradas en GitHub pues también se han resuelto issues en DefectDojo en local en cada ordenador de cada miembro en paralelo con GitHub.

2. Manual de despliegue y uso

2.1 Requisitos previos

- Git con acceso al repositorio PAI-4-Charmaway.
- Docker Engine 24+ y Docker Compose v2.
- Python 3.12 (sólo si se desea ejecutar la aplicación o las herramientas fuera de Docker).
- Cuenta GitHub con permisos sobre el repositorio para los workflows y la creación de tokens.

2.2 Despliegue de la aplicación Charmaway

```
git clone https://github.com/PAI-4-Charmaway/PAI-4-Charmaway.git
cd PAI-4-Charmaway && python -m venv .venv && source .venv/Scripts/activate
pip install -r requirements.txt
cp charmaway/.env.example charmaway/.env
cd charmaway && python manage.py migrate && python manage.py runserver 8080
```

2.3 Despliegue de DefectDojo en local

El stack de DefectDojo está parametrizado en defectdojo/docker-compose.yml. Para levantarlo:

```
cd defectdojo && docker compose up -d
```

El primer arranque tarda entre tres y cinco minutos, ya que el servicio initializer ejecuta las migraciones y carga los datos semilla. Una vez listo, la interfaz web queda disponible en <http://localhost:8080> con las credenciales admin/admin (deben cambiarse antes de cualquier exposición pública).

Obtención de la API key necesaria para sync_issues.py:

```
curl -X POST http://localhost:8080/api/v2/api-token-auth/ -H "Content-Type: application/json" -d '{"username":"admin","password":"admin"}'
```

Para detener el stack:

```
docker compose down          # mantiene los volúmenes
docker compose down -v       # elimina también la base de datos
```

2.4 Configuración del pipeline

El workflow `.github/workflows/devsecops.yml` requiere dos secrets en Settings → Secrets and variables → Actions para la subida automática a DefectDojo:

Secret	Descripción
DEFECTDOJO_URL	URL pública del DefectDojo. Durante el desarrollo se ha empleado un túnel ngrok contra la instancia local del equipo encargado del Dojo.
DEFECTDOJO_API_KEY	API key generada en el panel de DefectDojo del usuario admin.

Si los secrets no están definidos los pasos de subida se omiten silenciosamente (marcados como non-blocking) y el pipeline continúa generando los artifacts descargables.

2.5 Ejecución y verificación

Lanzamiento manual del pipeline a demanda contra cualquier rama:

```
gh workflow run devsecops.yml --ref <rama>
```

Descarga de reportes una vez finalizado:

```
gh run download <run-id>
```

Sincronización automática de findings a issues de GitHub (requiere `.env` en la raíz con `DOJO_URL`, `DOJO_TOKEN`, `GITHUB_API` y `GITHUB_TOKEN`):

```
python scripts/sync_issues.py
```

2.6 Errores comunes



Síntoma	Solución
Puerto 8080 ya en uso al levantar DefectDojo	Localizar el proceso (<code>netstat -ano grep :8080</code>) y cerrarlo, o crear un <code>docker-compose.override.yml</code> exponiendo otro puerto.
DefectDojo upload failed (non-blocking) en CI	El túnel ngrok que expone el Dojo del equipo está caído. Verificar el estado del túnel o ignorar el aviso si solo se necesitan los artifacts.
pip-audit reporta vulnerabilidades resueltas	Volver a ejecutar el <code>reimport-scan</code> ; DefectDojo marcará automáticamente los findings ausentes como Mitigated.
Issue duplicada al ejecutar <code>sync_issues.py</code>	Comprobar que el finding tiene el tag <code>enviado-a-github</code> en DefectDojo; si no, el script lo creará de nuevo.

CONFIDENCIAL

3. Grado de completitud

La siguiente tabla traza, objetivo a objetivo, el grado de cumplimiento alcanzado y la evidencia concreta verificable en el repositorio. Se distinguen los objetivos completados íntegramente de aquéllos cubiertos sólo parcialmente, indicando en cada caso el motivo.

#	Objetivo del enunciado	Grado	Evidencia
1	Definir un pipeline CI/CD en un repositorio SCM.	Completo	Workflow GitHub Actions en <code>.github/workflows/devsecops.yml</code> con triggers automáticos (push a develop/main) y manual (workflow_dispatch).
2	Seleccionar al menos tres herramientas de pruebas de seguridad.	Completo	Cuatro herramientas integradas (supera el mínimo): pip-audit (SCA), Bandit (SAST), Trivy (IaC) y OWASP ZAP (DAST).
3	Integrar las herramientas en el ciclo de vida del desarrollo.	Completo	Cada job del workflow ejecuta su herramienta, publica el reporte como artifact y lo envía a DefectDojo. La integración está versionada en el repositorio y se ejecuta sin intervención manual en cada push.
4	Desarrollar tests/pruebas que detecten vulnerabilidades en cada etapa del pipeline.	Parcial	Las cuatro herramientas detectan vulnerabilidades reales (4 CVEs en SCA, 339 en SAST, 12 alertas en DAST). Se han configurado herramientas existentes en lugar de desarrollar tests propios; se considera la interpretación más razonable del objetivo dado el alcance académico.
5	Seleccionar una herramienta de gestión de vulnerabilidades.	Completo	DefectDojo desplegado en <code>defectdojo/docker-compose.yml</code> . Productos, engagements y findings creados automáticamente desde CI vía <code>reimport-scan</code> . Sincronización con GitHub Issues mediante <code>scripts/sync_issues.py</code> .

3.1 Evidencias visuales

Platform
Solutions
Resources
Open Source
Enterprise
Pricing
Search or jump to...
Sign in
Sign up

PAI-5-Charmaway / PAI-5-Charmaway
Public
Notifications
Fork
Star

Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights

DevSecOps
DevSecOps #23
Sign in to view logs

Summary
All jobs
Run details
Usage
Workflow file

Manually triggered 49 minutes ago
Status: Success
Total duration: 1m 46s
Artifacts: 5

devsecops.yml
on: workflow_dispatch

- SCA - pip-audit 26s
- SAST - Bandit 8s
- laC - Trivy 10s
- DAST - OWASP ZAP 1m 42s

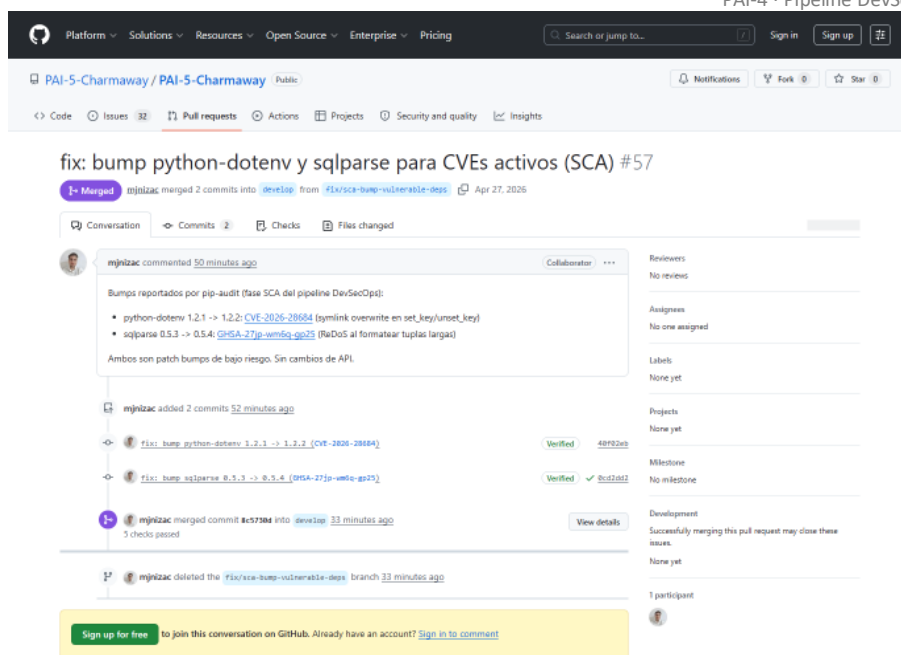
Annotations
4 warnings

- SAST - Bandit: Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actio... Show more
- laC - Trivy: Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actio... Show more
- SCA - pip-audit: Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actio... Show more
- DAST - OWASP ZAP: Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actio... Show more

Artifacts
Produced during runtime

Name	Size	Digest
bandit-report	14.3 KB	sha256:7781c2a52f5b8f85c870eb28c2479448e...
pip-audit-report	1.19 KB	sha256:6228b0bdf8832f748b1a8381e75b0252...
trivy-report	3.43 KB	sha256:38222748f1bd28b2a7e5ab0146475e7e54...
zap-report	31.8 KB	sha256:0632639ef93b0a3a5d3b4e7b7b44908ff...
zap-xml-report	6.52 KB	sha256:c28cb48dec89e99c28eaf23bc0c322c472...

Figura 2. Workflow run 24991055473 con las cuatro fases del pipeline en verde y los cinco artifacts publicados.



fix: bump python-dotenv y sqlparse para CVEs activos (SCA) #57

Merged mjnizac merged 2 commits into `develop` from `fix/sca-bump-vulnerable-deps` Apr 27, 2025

Conversation · Commits (2) · Checks · Files changed

mjnizac commented 50 minutos ago

Bumps reportados por pip-audit (fase SCA del pipeline DevSecOps):

- python-dotenv 1.2.1 -> 1.2.2 (CVE-2025-28664) (symlink overwrite en set_key/unset_key)
- sqlparse 0.5.3 -> 0.5.4 (GHSA-27jp-wmfc-gp23) (RaDoS al formatear tuplas largas)

Ambos son patch bumps de bajo riesgo. Sin cambios de API.

mjnizac added 2 commits 52 minutos ago

- fix: bump python-dotenv 1.2.1 -> 1.2.2 (CVE-2826-28664) **Verified** 4892ab
- fix: bump sqlparse 0.5.3 -> 0.5.4 (GHSA-27jp-wmfc-gp23) **Verified** 8cd2d62

mjnizac merged commit `8cd2d62` into `develop` 33 minutos ago

3 checks passed

mjnizac deleted the `fix/sca-bump-vulnerable-deps` branch 33 minutos ago

Sign up for free to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Figura 3. Pull Request #57 con los dos commits referenciando sus identificadores CVE/GHSA.

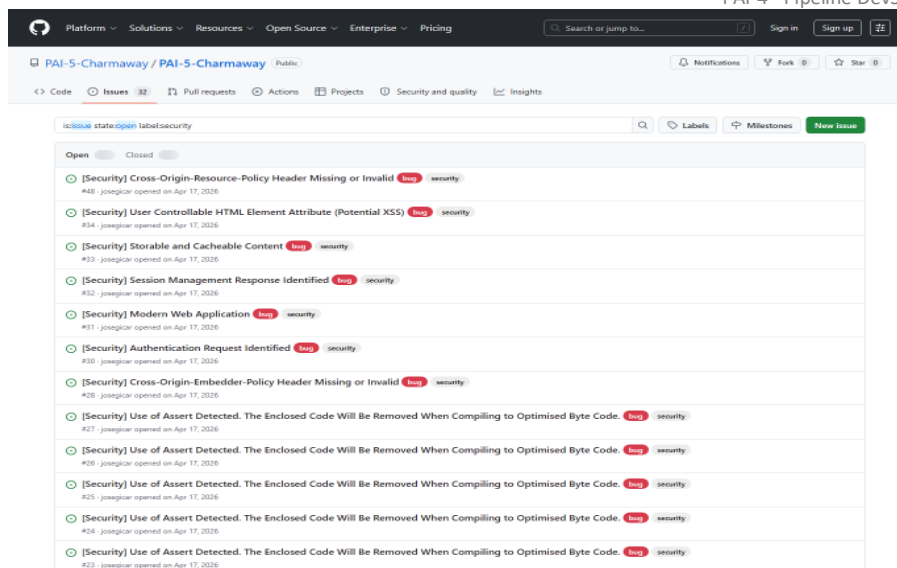


Figura 4. Listado de 32 issues de seguridad abiertos automáticamente desde DefectDojo.

DefectDojo · Estado de findings tras la mitigación

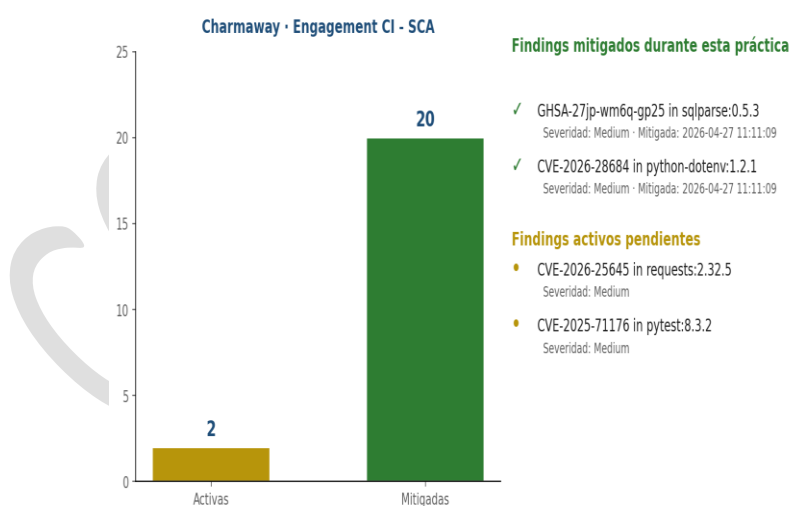


Figura 5. Panel de DefectDojo mostrando los findings activos y mitigados de la fase SCA.

3.2 Evidencias del flujo end-to-end

- PR #57 (a develop) y PR #58 (a main): bumps de python-dotenv y sqlparse, rebase merge para preservar historial individual de cada commit con su CVE en el mensaje.
- Commits 043ab9f y fc9aa9b en main, cada uno cita su identificador CVE/GHSA en el mensaje.
- Workflow run 24991055473: las cuatro fases del pipeline en verde tras el fix.

- DefectDojo: 4 → 2 findings activos en SCA, dos vulnerabilidades marcadas como Mitigated con timestamp 2026-04-27T11:11:09Z.
- 36 issues abiertas automáticamente en GitHub desde DefectDojo a través de sync_issues.py, sirviendo de backlog priorizable para el equipo.

3.3 Limitaciones conocidas

- Los hallazgos SAST ubicados en código de tests (assert_used, hardcoded_password_funcarg) no se han mitigado: corresponden a fixtures de pruebas (charmaway/administrator/tests.py, charmaway/catalog/tests.py) y se aceptan tras revisión, dado que Bandit no aplica reglas distintas a código de test por defecto. Quedan registrados en DefectDojo para revisión futura.
- De los 4 CVEs detectados por SCA se han mitigado dos. Los dos restantes (pytest, requests) requieren un mayor bump y se han pospuesto al cierre de la práctica.
- La instancia de DefectDojo se aloja en el equipo de un miembro del grupo y se expone vía túnel ngrok, lo que provoca pérdidas puntuales de subida cuando el túnel está caído. Los artifacts del pipeline siempre quedan disponibles en GitHub Actions como respaldo.
- ZAP se ejecuta en modo Baseline (escaneo pasivo). Una cobertura más exhaustiva exigiría un escaneo Active, que excede el tiempo razonable de un job de CI por commit.

3.4 Conclusión

Se han cumplido los cinco objetivos del enunciado, con cobertura superior al mínimo exigido (cuatro herramientas frente a tres) y evidencia verificable de un ciclo completo detección → ticketing → corrección → verificación, sostenido sobre infraestructura reproducible mediante docker-compose y workflows declarativos en YAML. El cuarto objetivo se considera parcialmente cubierto en el sentido literal del enunciado, ya que se han integrado herramientas existentes en lugar de desarrollar tests propios; la integración resultante detecta vulnerabilidades reales en cada fase del pipeline, lo que cubre la finalidad subyacente.